# The Active Inference Framing of the Decision Loop

`src/template_formal/agent/agent.py`'s `Agent.decide` scores each candidate action by a closed-form quantity, formalized here once and cited by number everywhere else in this manuscript that uses it:

### Definition (Expected free energy of a candidate action)

For a scalar Gaussian belief $Q(o \mid \text{action}) = \mathcal{N}(\mu_q, \sigma_q^2)$ about the outcome a candidate action would produce, and a fixed Gaussian preference $P(o) = \mathcal{N}(\mu_p, \sigma_p^2)$, subject to $\sigma_q^2 > 0$ and $\sigma_p^2 > 0$ (enforced at construction by `BeliefState.__post_init__`, ISC-81), the *expected free energy* of the action is the sum of a KL-divergence risk term and a differential-entropy ambiguity term over $Q$, given in `eq:expected-free-energy`. `Agent.decide` selects the candidate action minimizing this quantity.

## The decision loop I

$$G(\text{action}) = \mathrm{KL}[Q(o \mid \text{action}) \,\|\, P(o)] + \mathrm{H}[Q(o \mid \text{action})] \tag{1}$$

Both terms of eq. 1 are ordinary, exactly-computable closed forms — the univariate Gaussian KL-divergence and the univariate Gaussian differential entropy — and `tests/agent/test_agent_free_energy.py` checks the implementation against a hand-derived numeric expectation (ISC-29), not merely a "runs without error" smoke assertion.

A third adversarial pass (FirstPrinciples and an independent security review, converging on the same finding without coordinating) caught a real gap here: `BeliefState.variance` — the divisor in eq. 1's KL term and the argument to $\log$ in its entropy term — carried no validation, so `BeliefState(mean=0.0, variance=0.0)` constructed silently and only failed three calls later with an undocumented `ZeroDivisionError`. `BeliefState` now validates `variance` (finite, `> 0.0`) at construction

(ISC-81), mirroring the same construction-time-guard pattern the storage layer's SQL identifiers and the colony harness's `decay/sensing_noise_std` already use. `tests/agent/test_agent_free_energy.py` proves the fix is load-bearing: the `ValueError` now fires at `BeliefState.__init__`, not deep inside $G$'s computation.

## What this borrows from Friston (2005), and what it does not I

The general framing — that adaptive behavior can be cast as approximate minimization of a variational free energy over beliefs about hidden or observed states — traces to Friston (2005), "A Theory of Cortical Responses." That paper establishes the free-energy principle for perception; it does not itself present the risk/ambiguity decomposition of *expected* free energy used above, which was formalized in later active-inference literature building on it. This template borrows only the general framing — behavior as free-energy minimization over a belief distribution — and states the borrowed formula explicitly in the source docstring rather than presenting it as a verbatim reproduction of Friston's 2005 equations. What a research-grade active-inference implementation would additionally require — hierarchical generative models, policy-conditioned rollouts over multi-step futures, precision-weighting of prediction errors, and

## What this borrows from Friston (2005), and what it does not I

message-passing (variational) inference over a structured generative model — is out of scope for this template and is not claimed to be present.

## From individual free-energy minimization to collective organization I

The colony's convergence behavior (sec. 6; not claimed as emergence in the stronger sense there defined) is not produced by any single agent's free-energy calculation; it is produced by many agents independently minimizing $G$ against a **shared, environment-mediated** substrate — the pheromone field (`src/template_formal/colony/pheromone.py`). This is exactly the bridge Ehresmann and Vanbremeersch's Memory Evolutive Systems framework (Ehresmann and Vanbremeersch (2007)) is built to describe: a hierarchy in which lower-level components (individual agents, each running its own local free-energy minimization) interact through a shared structure to produce higher-level, emergent organizational patterns, formalized category-theoretically as colimits over evolving diagrams of interacting sub-systems. This template does not implement or check Ehresmann and Vanbremeersch's formal colimit construction — the connection drawn

## From individual free-energy minimization to collective organization I

here is, like sec. 4's functorial framing, a conceptual bridge from an individual-level computational mechanism (expected-free-energy minimization) to a collective-level phenomenon (stigmergic convergence), not a claim that this template's code constitutes a formal Memory Evolutive System.

## Structural boundary the decision loop respects I

`Agent.decide` and `Agent.record_observation` never read or write another agent's storage file; the type system and the `Agent` class's public surface make this structurally true rather than merely conventionally true (ISC-30, sec. 4). The colony coordinator that drives many agents through repeated ticks holds references only to each agent's public interface and to the shared `PheromoneField Protocol` — never to an agent's internal `Database` or protocol `SessionEndpoint` (ISC-34). This keeps the "many local free-energy minimizers coordinating through a shared environment, with no shared internal state" framing honest at the code level, not just at the level of the manuscript's prose.